

Arquitectura de Microservicios en Producción: Caso Netflix

FORM-IC-01 — Investigación Individual | Unidad 3: Arquitecturas Distribuidas en la Industria

Estudiante: Vinueza Sánchez Harold Nicolás
Docente: Ing. Guerrero Ulloa Gleiston Cicerón
Institución: Universidad Técnica Estatal de Quevedo (UTEQ)
Ciclo: Aplicaciones Distribuidas — 2026–2027 PPA

1. Descripción del Caso: Netflix y su Transición a Microservicios

Netflix es hoy la plataforma de *streaming* de video más grande del mundo, con más de 230 millones de suscriptores en más de 190 países y picos de tráfico que representan cerca del 15 % del ancho de banda global de Internet. Hasta 2007, toda su infraestructura residía en un sistema monolítico alojado en centros de datos propios; la totalidad de la lógica de negocio —reproducción, recomendaciones, facturación, autenticación— estaba comprimida en una sola base de código que se desplegaba como una unidad indivisible.

En 2008, una corrupción crítica en la base de datos central interrumpió los envíos de DVD durante tres días consecutivos, exponiendo de forma contundente la fragilidad del modelo centralizado ante fallos puntuales. Este incidente motivó la toma de decisión más relevante en la historia de la ingeniería de software de la compañía: migrar progresivamente hacia Amazon Web Services (AWS) y descomponer el monolito en **microservicios** independientes. La transición, que demandó aproximadamente siete años, quedó prácticamente completada hacia 2015 [1].

Hoy, la plataforma opera con más de **700 microservicios** en producción. Entre los más críticos se encuentran el servicio de recomendación personalizada, el de codificación y transcodificación de vídeo (plataforma *Cosmos*), el de autenticación OAuth, el de gestión de perfiles y el de facturación recurrente. Cada uno es propiedad de un equipo pequeño (“two-pizza team”) que gestiona su ciclo de vida completo: diseño, desarrollo, pruebas, despliegue y monitoreo [2]. Esta estructura organizacional, conocida como DevOps descentralizado, es una consecuencia directa de la arquitectura adoptada [1].

2. Por qué Eligieron la Arquitectura de Microservicios

La decisión de adoptar microservicios en Netflix no fue ideológica sino pragmática, y respondió a tres limitaciones estructurales del monolito que se volvieron insostenibles con el crecimiento exponencial del servicio:

- 1. Límite de escalabilidad horizontal.** En la arquitectura monolítica, escalar un único componente (p. ej., el motor de recomendaciones) implicaba replicar la totalidad del sistema, lo que resultaba en un uso ineficiente de recursos y en costos operativos prohibitivos a escala global [4].
- 2. Velocidad de entrega comprometida.** Un cambio mínimo en cualquier módulo obligaba a redespargar la aplicación completa, con ventanas de mantenimiento superiores a seis horas y tasas de reversión (rollback) elevadas. Los microservicios eliminan este acoplamiento al permitir que cada servicio tenga su propio pipeline de integración y despliegue continuo (CI/CD) [1].
- 3. Riesgo de fallos en cascada.** En un sistema monolítico, la falla de un subcomponente no crítico podía derribar la experiencia completa del usuario. Los microservicios permiten aislar dominios de negocio de modo que los fallos se contienen y no se propagan al sistema global; principio formalizado como *bulkhead pattern* [3].

Adicionalmente, Netflix necesitaba una arquitectura que soportara el ciclo de innovación continua

característico de la industria del entretenimiento: nuevas características (reproducción sin conexión, co-visualización, plan con anuncios) debían lanzarse en semanas, no en meses [2].

3. Beneficios Obtenidos

La migración produjo mejoras sustanciales y cuantificables en tres ejes principales:

Escalabilidad independiente y eficiencia de recursos. Cada servicio escala en función exclusiva de su propia demanda. El servicio de codificación *Cosmos* puede procesar millones de activos multimedia sin incrementar los recursos del servicio de autenticación, que opera con patrones de carga completamente distintos. Esto redujo significativamente el costo de infraestructura respecto al modelo monolítico [2]. Estudios empíricos sobre arquitecturas comparadas confirman que la escalabilidad granular de los microservicios supera al monolito en escenarios de carga heterogénea [4].

Despliegue continuo y autonomía de equipos. La plataforma de Netflix realiza entre 4 000 y 6 000 despliegues por día en producción. Cada equipo lleva un servicio de *commit* a producción en pocas horas mediante pipelines automatizados. Esta capacidad es habilitada directamente por el desacoplamiento arquitectónico de los microservicios [1].

Resiliencia demostrada mediante ingeniería del caos. Netflix diseñó el *Chaos Monkey*, una herramienta que inyecta fallos aleatorios en servicios de producción para validar que el sistema tolera las interrupciones sin degradar la experiencia del usuario. Esta práctica, hoy extendida a toda la industria bajo el nombre de *chaos engineering*, fue posible precisamente porque los microservicios proveen el aislamiento necesario para absorber fallos individuales sin propagarlos [3].

4. Desafíos Encontrados

La adopción de microservicios introdujo una clase de complejidad inexistente en el monolito original. Los principales desafíos documentados en la literatura y confirmados por la experiencia de Netflix son:

- **Latencia y sobrecarga de red.** Toda interacción entre servicios que antes era una llamada en memoria se convierte en una petición de red (HTTP/gRPC), lo que introduce latencia medible y requiere estrategias de tolerancia como *circuit breakers*, *timeouts* y *retries* [3].
- **Observabilidad distribuida.** Con más de 700 servicios, identificar la causa raíz de un error exige rastreo distribuido (*distributed tracing*), correlación de *logs* entre múltiples instancias y métricas unificadas. Netflix desarrollaron herramientas como *Zipkin* y *Atlas* para cubrir esta necesidad [1].
- **Consistencia de datos eventual.** Cada microservicio gestiona su propia base de datos, lo que elimina las transacciones ACID globales. Garantizar la consistencia entre servicios exige patrones como *sagas* y mecanismos de compensación transaccional [4].
- **Complejidad operacional.** Orquestar el descubrimiento de servicios, balanceo de carga, gestión de secretos y actualizaciones sin tiempo de inactividad en cientos de microservicios demanda una plataforma de infraestructura madura (Kubernetes, Spinnaker, Consul) [2].

5. Aplicación al Proyecto Propio

Las lecciones del caso Netflix son directamente aplicables al proyecto de la asignatura Aplicaciones Distribuidas, que contempla el diseño de un **sistema de gestión académica distribuido**. A partir del análisis del caso, se identifican las siguientes decisiones de diseño:

1. **Descomposición por dominio.** Cada función de negocio —matrícula, calificaciones, notificaciones, autenticación— se implementa como un microservicio independiente con su propia base de datos, siguiendo el principio de *bounded context* de Domain-Driven Design [3].
2. **API Gateway centralizado.** Un Gateway único (p. ej., Kong o AWS API Gateway) recibe todas las peticiones externas, gestiona autenticación, limitación de tasa (*rate limiting*) y enrutamiento, simplificando la interfaz del cliente respecto a la complejidad interna del sistema.
3. **Comunicación asíncrona para operaciones no críticas.** Procesos como el envío de notificaciones o la generación de reportes PDF se desacoplan del flujo principal mediante un *message broker* (RabbitMQ o Kafka), mejorando la capacidad de respuesta del sistema y su tolerancia a picos de carga [1].
4. **Granularidad controlada.** La lección más importante del caso Netflix es que los microservicios *demasiado finos* generan una red de dependencias que anula los beneficios del desacoplamiento. Para el proyecto académico se propone iniciar con 4–5 servicios bien definidos y refinarlos conforme evolucione el sistema [4].

References

- [1] A. Balalaie, A. Heydarnoori, and P. Jamshidi, “Microservices Architecture Enables DevOps: Migration to a Cloud-Native Architecture,” *IEEE Software*, vol. 33, no. 3, pp. 42–52, May/Jun. 2016. DOI: [10.1109/MS.2016.64](https://doi.org/10.1109/MS.2016.64)
- [2] F. S. Miguel, N. Mareddy, A. K. Moorthy, and X. Liu, “Microservices for Multimedia: Video Encoding,” in *Proc. 1st ACM Mile-High Video Conf. (MHV)*, Denver, CO, USA, 2022. DOI: [10.1145/3510450.3517298](https://doi.org/10.1145/3510450.3517298)
- [3] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, Today, and Tomorrow,” in M. Mazzara and B. Meyer (Eds.), *Present and Ulterior Software Engineering*, Springer, Cham, 2017, pp. 195–216. DOI: [10.1007/978-3-319-67425-4_12](https://doi.org/10.1007/978-3-319-67425-4_12)
- [4] G. Blinowski, A. Ojdowska, and A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357–20374, 2022. DOI: [10.1109/ACCESS.2022.3152803](https://doi.org/10.1109/ACCESS.2022.3152803)